LSTAT2185: Final Porject

Teacher : *Pr. Eugen Pircalabelu*

Authors :
Devaux Nathan
Piron Louis-David

January 12, 2025

PART I.

Exercise 1

Consider the function f defined as:

$$f : \mathbb{R}_+^* \times \mathbb{R}_+^* \to \mathbb{R}, \quad (x, y) \mapsto x \log x + y \log y.$$

where $\mathbb{R}_+^* = (0, +\infty)$.

(1.a)

The function $f(x, y) = x \log x + y \log y$ is differentiable with respect to both x and y. We are thus able to compute the gradient $\nabla f = (\frac{\partial}{\partial x} f, \frac{\partial}{\partial y} f)^\top$, where :

$$\frac{\partial}{\partial x} f = \log x + 1 \quad \text{and} \quad \frac{\partial}{\partial y} f = \log y + 1$$

Therefore, the gradient ∇f at $a = (u, v)^\top$ is:

$$\nabla f(x, y)\Big|_{a=(u,v)^\top} = \begin{pmatrix} \log u + 1 \\ \log v + 1 \end{pmatrix}.$$

(1.b)

The Hessian matrix H is the matrix of second-order partial derivatives. The second-order partial derivative with respect to x is given by :

$$\frac{\partial^2 f}{\partial x^2} = \frac{d}{dx}(\log x + 1) = \frac{1}{x}.$$

Following the same logic, $\frac{\partial^2 f}{\partial y^2} = \frac{1}{y}$. The mixed partial derivatives are both equal to 0. We obtain therefore that the Hessian Matrix H at $a = (u, v)^\top$ is:

$$H = \begin{pmatrix} \frac{1}{u} & 0 \\ 0 & \frac{1}{v} \end{pmatrix}.$$

(1.c)

The Taylor expansion of $f(x, y) = x \log x + y \log y$ at the point $a = (u, v)^\top$ up to second order is given by:

$$f(x, y) \approx f(u, v) + \nabla f(u, v)^\top \begin{pmatrix} x - u \\ y - v \end{pmatrix} + \frac{1}{2} \begin{pmatrix} x - u \\ y - v \end{pmatrix}^\top H(u, v) \begin{pmatrix} x - u \\ y - v \end{pmatrix}$$

Substituting the values of $f(u, v)$, $\nabla f(u, v)$, and $H(u, v)$, we get:

$$f(x, y) \approx u \log u + v \log v + (\log u + 1)(x - u) + (\log v + 1)(y - v) + \frac{1}{2} \left(\frac{(x - u)^2}{u} + \frac{(y - v)^2}{v} \right)$$

$$f(x,y) \approx u\log u + v\log v + x\log u + x - u\log u - u + y\log v + y - v\log v - v + \frac{1}{2}\left(\frac{(x-u)^2}{u} + \frac{(y-v)^2}{v}\right)$$

$$f(x,y) \approx x\log u + y\log v + x - u + y - v + \frac{1}{2}\left(\frac{x^2 - 2xu + u^2}{u} + \frac{y^2 - 2yv + v^2}{v}\right)$$

$$f(x,y) \approx x\log u + y\log v + x - u + y - v + \frac{x^2}{2u} - x + \frac{u}{2} + \frac{y^2}{2v} - y + \frac{v}{2}$$

$$f(x,y) \approx x\log u + y\log v - \frac{u}{2} - \frac{v}{2} + \frac{x^2}{2u} + \frac{y^2}{2v}$$

(1.d)

We want to find the global minimum of $f(x,y) = x\log x + y\log y$ on $\mathbb{R}_+^* \times \mathbb{R}_+^*$. To do so, we begin by setting the gradient equal to zero :

$$\log x + 1 = 0 \quad \text{and} \quad \log y + 1 = 0$$

Solving these gives the critical point $(x^*, y^*)^\top = (e^{-1}, e^{-1})^\top$. This point **could be** a local minimum. However, as the Hessian matrix at the critical point is PD[1], $(e^{-1}, e^{-1})^\top$ **is** a local minimum :

$$H(e^{-1}, e^{-1}) = \begin{pmatrix} e & 0 \\ 0 & e \end{pmatrix}$$

If, we now prove that f is convex on $\mathbb{R}_+^* \times \mathbb{R}_+^*$, we prove that $(e^{-1}, e^{-1})^\top$ is a global minimum of f.

Based on **Theorem 7** (*cf.* page 28 of the syllabus), we know that the twice differentiable function $f : S \to \mathbb{R}$, with non-empty convex set S, is convex if and only if $H(x)$ is SDP $\forall x \in S$.

First, we need to verify that the domain of f, which is $\mathbb{R}_+^* \times \mathbb{R}_+^*$, is a convex set. According do **Definition 15** (*cf.* page 25 of the syllabus), a set $S \subseteq \mathbb{R}^n$ is convex if for any two points $a, b \in S$ and scalar $\lambda \in [0,1]$, the combination $\lambda a + (1-\lambda)b \in S$.

Specifically, for any two points $a = (x_1, y_1)^\top, b = (x_2, y_2)^\top \in \mathbb{R}_+^* \times \mathbb{R}_+^*$, we have:

$$\lambda \begin{pmatrix} x_1 \\ y_1 \end{pmatrix} + (1-\lambda)\begin{pmatrix} x_2 \\ y_2 \end{pmatrix} = \begin{pmatrix} \lambda x_1 + (1-\lambda)x_2 \\ \lambda y_1 + (1-\lambda)y_2 \end{pmatrix}$$

$\lambda x_1 + (1-\lambda)x_2 > 0$ and $\lambda y_1 + (1-\lambda)y_2 > 0$, confirming that the point lies in $\mathbb{R}_+^* \times \mathbb{R}_+^*$. Therefore, **the domain is convex**.

[1] all its eigenvalues are strictly positive. In this case, the eigenvalues are equal to $e > 0$.

We now check whether the Hessian matrix is PSD.

$H = \begin{pmatrix} \frac{1}{x} & 0 \\ 0 & \frac{1}{y} \end{pmatrix}$ is diagonal, so its eigenvalues are simply the diagonal elements :

$$\lambda_1 = \frac{1}{x} \quad \lambda_2 = \frac{1}{y}$$

As $x > 0$ and $y > 0$, these eigenvalues are strictly positive, which means that the Hessian matrix is PD. This implies that the function is **strictly convex**, which is stronger than convex.

In conclusion, based on **Theorem 7**, since f is twice differentiable and its Hessian is PD $\forall (x, y) \in \mathbb{R}_+^* \times \mathbb{R}_+^*$, we conclude that f is strictly convex on $\mathbb{R}_+^* \times \mathbb{R}_+^*$. And therefore conclude that $(e^{-1}, e^{-1})^\top$ is a global minimizer.

(1.e)

Newton Method :

Newton's update rule is :

$$x_{t+1} = x_t - H(x_t)^{-1} \nabla f(x_t)$$

Let us say that we take as starting point $x_0 = (e, e)^\top$, we have for the first step :

$$x_1 = x_0 - H(x_0)^{-1} \nabla f(x_0)$$

Where $H(x_0) = \begin{pmatrix} \frac{1}{e} & 0 \\ 0 & \frac{1}{e} \end{pmatrix} = \frac{1}{e} I_2 \implies H(x_0)^{-1} = e I_2$ and $\nabla f(x_0) = \begin{pmatrix} \log e + 1 \\ \log e + 1 \end{pmatrix} = \begin{pmatrix} 2 \\ 2 \end{pmatrix}$

Hence, the first update step is:

$$x_1 = \begin{pmatrix} e \\ e \end{pmatrix} - e \begin{pmatrix} 2 \\ 2 \end{pmatrix} = \begin{pmatrix} e - 2e \\ e - 2e \end{pmatrix} = \begin{pmatrix} -e \\ -e \end{pmatrix}$$

This point takes us outside of the domain of f and ∇f, showcasing Newton's sensitivity to the starting point.

If, as suggested in the email, we had taken $x_0 = (1, 1)^\top$, the first iteration would have resulted in the point $x_1 = (0, 0)^\top$. This point also lies outside the domain of f and ∇f. Moreover, we encounter another problem: $H(x_1)$ is non-invertible at this point. This highlights a second critical limitation of Newton's method.

Let'us try another sarting point $x_0 = (0.5, 0.5)^\top$:

$$H(x_0) = \begin{pmatrix} 2 & 0 \\ 0 & 2 \end{pmatrix} \implies H(x_0)^{-1} = \begin{pmatrix} \frac{1}{2} & 0 \\ 0 & \frac{1}{2} \end{pmatrix} \text{ and } \nabla f(x_0) = \begin{pmatrix} \log 0.5 + 1 \\ \log 0.5 + 1 \end{pmatrix} = \begin{pmatrix} 0.3069 \\ 0.3069 \end{pmatrix}$$

Applying the update formula:

$$x_1 = \begin{pmatrix} 0.5 \\ 0.5 \end{pmatrix} - \begin{pmatrix} \frac{1}{2} & 0 \\ 0 & \frac{1}{2} \end{pmatrix} \begin{pmatrix} 0.3069 \\ 0.3069 \end{pmatrix} = \begin{pmatrix} 0.5 \\ 0.5 \end{pmatrix} - \begin{pmatrix} 0.15345 \\ 0.15345 \end{pmatrix} = \begin{pmatrix} 0.34655 \\ 0.34655 \end{pmatrix}$$

The Hessian and gradient at $x_1 = (0.34655, 0.34655)^\top$ are:

$$H(x_1) = \begin{pmatrix} \frac{1}{0.34655} & 0 \\ 0 & \frac{1}{0.34655} \end{pmatrix} = \begin{pmatrix} 2.886 & 0 \\ 0 & 2.886 \end{pmatrix} \implies H(x_1)^{-1} = \begin{pmatrix} \frac{1}{2.886} & 0 \\ 0 & \frac{1}{2.886} \end{pmatrix}$$

$$\nabla f(x_1) = \begin{pmatrix} \log 0.34655 + 1 \\ \log 0.34655 + 1 \end{pmatrix} = \begin{pmatrix} -1.060 + 1 \\ -1.060 + 1 \end{pmatrix} = \begin{pmatrix} -0.060 \\ -0.060 \end{pmatrix}$$

Applying the update formula, we find $x_2 = (0.36737, 0.36737)^\top$. Note that we are already very close too the true minimum $(e^{-1}, e^{-1})^\top \approx (0.3678, 0.3679)^\top$.

Conjugate Gradient (CG) method.

To compare the CG method with Newton's method, we will apply the algorithm to $(e, e)^\top$.

CG's update rule is :

$$x_{t+1} = x_t + \alpha_t p_t$$

For the first iteration, $p_0 = -\nabla f(x)\big|_{x=x_0}$ leading to :

$$x_1 = x_0 - \alpha_0 \nabla f(x)\big|_{x=x_0}$$

By taking $x_0 = (e, e)^\top$ as starting point, we obtain :

$$x_1 = \begin{pmatrix} e \\ e \end{pmatrix} - \alpha_0 \begin{pmatrix} 2 \\ 2 \end{pmatrix}$$

Where $\alpha_0 = \arg\min_\alpha f\left(x_0 - \alpha \nabla f(x_0)\right)$. Let us note this function $\phi(\alpha)$. α_0 can be found by solving the equation $\phi'(\alpha) = 0$:

$$\phi'(\alpha) = 2\left[-2\log(e - 2\alpha) - 2\right] = -4\log(e - 2\alpha) - 4$$

Setting $\phi'(\alpha) = 0$ and exponentiating both sides:

$$\phi'(\alpha) = 0 \implies \log(e - 2\alpha) = -1$$

$$e - 2\alpha = e^{-1} \implies 2\alpha = e - e^{-1} \implies \alpha_0 = \frac{e - e^{-1}}{2}$$

Therfore, x_1 is :

$$x_1 = \begin{pmatrix} e \\ e \end{pmatrix} - \frac{e - e^{-1}}{2} \begin{pmatrix} 2 \\ 2 \end{pmatrix} = \begin{pmatrix} e - (e - e^{-1}) \\ e - (e - e^{-1}) \end{pmatrix} = \begin{pmatrix} e^{-1} \\ e^{-1} \end{pmatrix}.$$

At $x_1 = (e^{-1}, e^{-1})$, the gradient becomes:

$$\nabla f(e^{-1}, e^{-1}) = \begin{pmatrix} \log e^{-1} + 1 \\ \log e^{-1} + 1 \end{pmatrix} = \begin{pmatrix} -1 + 1 \\ -1 + 1 \end{pmatrix} = \begin{pmatrix} 0 \\ 0 \end{pmatrix}.$$

Since the gradient is zero, the method has converged, and no further updates are needed.

Therefore, $x_2 = x_1$. The CG method[2] converged faster and that it didn't led us out of the domain !

If, as suggested in the email, we had taken $x_0 = (1,1)^\top$, the first iteration would have resulted in the point $(e^{-1}, e-1)^\top$ in one iteration as well. Again, this is due to the step size optimization ($\phi'(\alpha_0) = 0 \implies \alpha_0 = 1 - e^{-1}$).

BFGS method.

To compare the BFGS method with Newton's method, we will apply the algorithm to the same two points as for Newton, namely $(e,e)^\top$ and $(0.5, 0.5)^\top$.

BFGS's update rule is :

$$x_{t+1} = x_t - (B_t)^{-1}\nabla f(x_t)$$

We set the initial approximation of the inverse Hessian $(B_0)^{-1}$ to the identity matrix, and compute x_1 :

$$x_1 = \begin{pmatrix} e \\ e \end{pmatrix} - \begin{pmatrix} 1 & 0 \\ 0 & 1 \end{pmatrix} \begin{pmatrix} 2 \\ 2 \end{pmatrix} = \begin{pmatrix} e-2 \\ e-2 \end{pmatrix}.$$

We now compute :

$$B_1 = B_0 - \frac{B_0 s s^\top B_0}{s^\top B_0 s} + \frac{y y^\top}{s^\top y}$$

Where :

$$s = x_1 - x_0 = \begin{pmatrix} e-2 \\ e-2 \end{pmatrix} - \begin{pmatrix} e \\ e \end{pmatrix} = \begin{pmatrix} -2 \\ -2 \end{pmatrix}$$

$$y = \nabla f(x_1) - \nabla f(x_0) = \begin{pmatrix} \log(e-2)+1 \\ \log(e-2)+1 \end{pmatrix} - \begin{pmatrix} 2 \\ 2 \end{pmatrix} \approx \begin{pmatrix} -1.33 \\ -1.33 \end{pmatrix}$$

As $B_0 = I$, we deduce:

$$s^\top s = 8, \quad s^\top y = 5.328, \quad s s^\top = \begin{pmatrix} 4 & 4 \\ 4 & 4 \end{pmatrix}, \quad y y^\top = \begin{pmatrix} (-1.33)^2 & (-1.33)^2 \\ (-1.33)^2 & (-1.33)^2 \end{pmatrix}$$

Thus:

$$B_1^{-1} = \begin{pmatrix} 1 & 0 \\ 0 & 1 \end{pmatrix} - \frac{1}{8}\begin{pmatrix} 4 & 4 \\ 4 & 4 \end{pmatrix} + \frac{1}{5.328}\begin{pmatrix} (-1.33)^2 & (-1.33)^2 \\ (-1.33)^2 & (-1.33)^2 \end{pmatrix}$$

[2]Strictly speaking, this does not constitute a full implementation of the conjugate gradient (CG) algorithm. Due to the step size optimization (line search), the algorithm converges in a single iteration, as the initial direction of descent happens to lead directly to the minimum for this specific problem.

$$B_1^{-1} \approx \begin{pmatrix} 0.833 & -0.167 \\ -0.167 & 0.833 \end{pmatrix}.$$

We now compute x_2 using the update rule:

$$x_2 = \begin{pmatrix} e-2 \\ e-2 \end{pmatrix} - \begin{pmatrix} 0.833 & -0.167 \\ -0.167 & 0.833 \end{pmatrix} \begin{pmatrix} 0.668 \\ 0.668 \end{pmatrix} \approx \begin{pmatrix} 0.218 \\ 0.218 \end{pmatrix}$$

Unlike Newton's method, which failed to converge when starting from $(e, e)^\top$ and $(1, 1)^\top$, CG converged to the true minimum $(e^{-1}, e^{-1})^\top$ in just 1 iteration. On the other hand, BFGS, while making progress, only approached the minimum after 2 iterations and had not yet fully converged.

(1.f)

Numerically, both CG and BFGS algorithms converge to the optimal point $(0.3678, 0.3678)^\top$, equivalent to the true minimum $(e^{-1}, e^{-1})^\top$. Again, CG converged faster than BFGS.

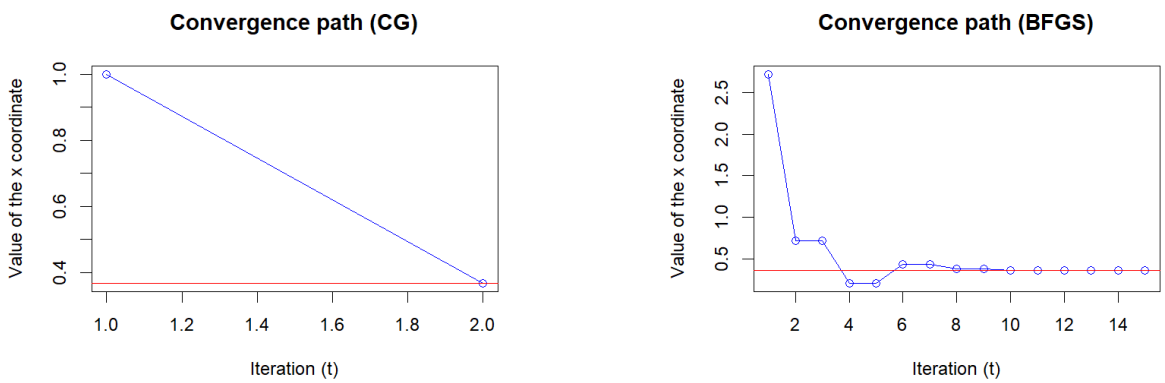

Figure 1: Convergence path of the x coordinate for BFGS and CG methods. The true minimum is represented by the red line.

(1.g)

When using Newton's method for the problem starting at $(1, 1)\top$, the key issue is that, at this point, the gradient and Hessian lead to updates that exit the domain of the objective function. By introducing a learing rate α, the step size can be controlled, which allow us to stain within the domain.This adjustment helps in avoiding the problem of overshooting or taking steps that move away from the feasible region. However, it is important to note that finding an appropriate learning rate can still be challenging, as it might require tuning for different problems or points in the optimization process.

In contrast, CG works only with first-order information (the gradient). As a result, it is less sensitive to issues with the conditioning of the Hessian and is more stable in such

situations. Moreover, as its formula takes into account the previous directions, it tends to balance out errors from one iteration to the next, which results in a more robust and predictable convergence path.

Instead of using a fixed learning rate, alternatives for improving Newton's method could include using line search techniques, which adjust the step size dynamically based on the objective function's behavior at each iteration. Additionally, quasi-Newton methods, such as BFGS, approximate the Hessian and update it iteratively, reducing the computational burden. These methods are less prone to instability than pure Newton's method and can be more efficient in high-dimensional settings.

Exercise 2

(2.a)

We are interested in modeling the data with a c.d.f of the form :

$$F(x) = \exp\left(-\left(1+\xi\frac{x-\mu}{\sigma}\right)^{-1/\xi}\right)$$

Where $\mu, \xi \in \mathbb{R}$ and $\sigma > 0$.

Let us first derive the log-likelihood function ℓ, which is defined as :

$$\ell(\mu,\sigma,\xi|x_i,\ldots,x_{22}) = \log\left(\prod_{i=1}^{22} f(x_i;\mu,\sigma,\xi)\right) = \sum_{i=1}^{22}\log f(x_i;\mu,\sigma,\xi)$$

To do so, we first need to derive the density function $f(x) = dF(x)/dx$:

$$f(x;\mu,\sigma,\xi) = \frac{1}{\sigma}\left(1+\xi\frac{x-\mu}{\sigma}\right)^{-(1/\xi+1)}\exp\left(-\left(1+\xi\frac{x-\mu}{\sigma}\right)^{-1/\xi}\right)$$

This function is also known as the density function of the *Generalized Extreme Value* (GEV) distribution, which is widely used in extreme value theory to model the distribution of maximum or minimum values in data sets.

Its log-likelihood is given by :

$$\ell(\mu,\sigma,\xi) = -n\log(\sigma) - \left(1+\frac{1}{\xi}\right)\sum_{i=1}^{n}\log\left(1+\xi\frac{x_i-\mu}{\sigma}\right) - \sum_{i=1}^{n}\left(1+\xi\frac{x_i-\mu}{\sigma}\right)^{-1/\xi}$$

And its gradient by $\nabla\ell = (\frac{\partial}{\partial\mu}\ell, \frac{\partial}{\partial\sigma}\ell, \frac{\partial}{\partial\xi}\ell)^\top$, with :

$$\frac{\partial}{\partial\mu}\ell = \frac{1+\xi}{\sigma}\sum_{i=1}^{22}\left(\frac{1}{1+\xi\frac{x_i-\mu}{\sigma}}\right) - \sum_{i=1}^{22}\frac{1}{\sigma}\left(1+\xi\frac{x_i-\mu}{\sigma}\right)^{-(1/\xi+1)}$$

$$\frac{\partial}{\partial\sigma}\ell = \frac{-n}{\sigma} + \frac{1+\xi}{\sigma^2}\sum_{i=1}^{22}\frac{x_i-\mu}{1+\xi\frac{x_i-\mu}{\sigma}} - \frac{1}{\sigma^2}\sum_{i=1}^{22}(x_i-\mu)\left(1+\xi\frac{x_i-\mu}{\sigma}\right)^{-(1/\xi+1)}$$

$$\frac{\partial}{\partial\xi}\ell = \frac{1}{\xi^2}\sum_{i=1}^{22}\log(1+\xi\frac{x_i-\mu}{\sigma}) + \left(\frac{-1}{\xi}-1\right)\sum_{i=1}^{22}\frac{x_i-\mu}{\sigma+\xi(x_i-\mu)}$$

$$-\sum_{i=1}^{22}\exp\left[\frac{-1}{\xi}\log(1+\xi\frac{\xi-\mu}{\sigma})\right]\left[\frac{1}{\xi^2}\log(1+\xi\frac{\xi-\mu}{\sigma}) - \frac{1}{\xi}\frac{x_i-\mu}{\sigma+\xi(x_i-\mu)}\right]$$

For the implementation part, we choose to use the `R` optimizer `Optim`. By default `Optim` perform minimization. Therefore, to find the maximum, we have to use the negative log-likelihood $\mathcal{L} = -\ell$ as well as its corresponding grandient $\nabla\mathcal{L} = -\nabla\ell$.

We decided to initialize our algorithm with the following parameters :

$$\theta_0 = (\mu_0 = 93.76, \sigma_0 = 13.17, \xi_0 = 0.1)^\top$$

For the location parameter μ and the scale parameter σ, the starting values are given by two natural candidates to start with : the mean and standard deviation (respectively). The shape parameter ξ can take several values. However, for the estimation of the GEV parameters, a common starting choice is 0.1.

Both BFGS and CG converge to the following θ^* :

$$\theta^* = (\mu^* = 87.52, \sigma^* = 9.57, \xi^* = 0.07)^\top$$

In the context of the GEV distribution, a parameter $\xi = 0.07$ suggests that the distribution belongs to the **Fréchet** family [3].

(2.b)

We now consider the simplified model of **Gumbel** ($\xi \to 0$):

$$F(x) = \exp\left(-\exp\left(-\frac{x-\mu}{\sigma}\right)\right)$$

For the corresponding density function, the log-likelihood is :

$$\ell(\mu, \sigma) = -n\log(\sigma) - \sum_{i=1}^{22}\left[\exp\left(-\frac{x_i-\mu}{\sigma}\right) + \frac{x_i-\mu}{\sigma}\right]$$

its gradient is given by $\nabla\ell = (\frac{\partial}{\partial\mu}\ell, \frac{\partial}{\partial\sigma}\ell)^\top$, with :

$$\frac{\partial}{\partial\mu}\ell = -\frac{1}{\sigma}\sum_{i=1}^{22}\left[\exp\left(-\frac{x_i-\mu}{\sigma}\right) - 1\right]$$

$$\frac{\partial}{\partial\sigma}\ell = -\frac{n}{\sigma}\sum\left[\frac{x_i-\mu}{\sigma^2}\cdot\left(1-\exp\left(-\frac{x_i-\mu}{\sigma}\right)\right)\right]$$

Again, for the implementation part, we use $\mathcal{L} = -\ell$ and $\nabla\mathcal{L} = -\nabla\ell$. After running the algorithm, we get the following θ^*:

$$\theta^*_{CG} = (\mu^* = 88.48, \sigma^* = 9.69)^\top \quad \theta^*_{BFGS} = (\mu^* = 88.6, \sigma^* = 9.59)^\top$$

[3] Note as well that when using the `gev.fit()` function from the `R` package `ismev`, which estimates the parameters of a GEV distribution through maximum likelihood estimation, we find the same parameter estimates, providing a validation of our calculations and implementation.

(2.c)

When we compare the empirical c.d.f. of the data to the theoretical c.d.f.s from Model 1 (Fréchet) and Model 2 (Gumbel), we notice that both models fit the data verry well. Model 1 seems to fit the data slightly better.

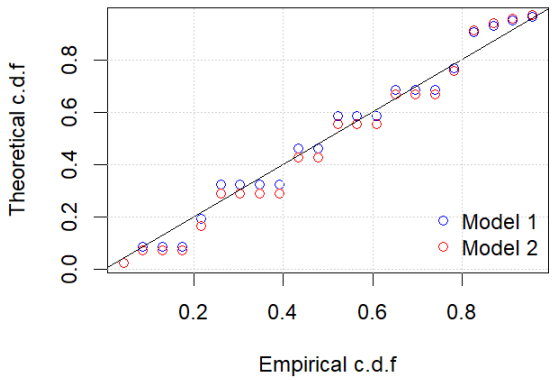

Figure 2: Model 1 corresponds to the Fréchet model found in question 2.a and Model 2 corresponds to the Gumbel model found in question 2.b

(2.d)

The AIC can be computed based on the following formula : $AIC = 2p - 2\ell$, where p is the number of parameters of the mode and ℓ its log-likelihood. When we perform these computation, we got the following results :

$$AIC_{\text{Fréchet}} = 2{\cdot}3 - 2{\cdot}(-85.3998) = 177.1189 \quad AIC_{\text{Gumbel}} = 2{\cdot}2 - 2{\cdot}(-85.55945) = 175.1189$$

As we usually prefer a model that minimizes the AIC, model 2 is slightly preferred. Even though model 1 offers a slightly higher log-likelihood (indicating a marginally better fit to the data, as noticed in 2.c), model 2 is preferred by the AIC due to its simplicity (1 fewer parameter) and comparable performance (log-likelihood).

If simplicity is a priority, model 2 remains the better choice. However, if the additional parameter in model 1 has meaningful significance, and particularly if it contributes to better capturing the tails of the distribution—a key consideration in extreme value analysis—it could justify selecting the more complex model despite the small AIC penalty.

PART II.

Exercise 3

(3.a)

For an i.i.d sample $\mathcal{X} = X_1, \ldots, X_n$ with $X_i \sim Weibull(\lambda, c)$ and c known, we are interrested in estimating the cth raw moment $\mathbb{E}[X^c]$. A natural estimator for this quantity is the empirical mean of X_i^c:

$$T(\mathcal{X}) = \theta(F_n) = \frac{1}{n}\sum_{i=1}^{n} X_i^c.$$

We choose this estimator because it is a **consistent estimator** (*i.e.* $T(\mathcal{X}) \xrightarrow{P} \mathbb{E}[X^c]$). This statistical property comes from the fact that the estimator is unbiased and that its variance $\to 0$ as $n \to \infty$.

Note as well that for a Weibull distribution, an exact formula for the kth raw moment is given by :

$$\mathbb{E}[X^k] = \lambda^c \Gamma\left(1 + \frac{c}{k}\right)$$

In the particular case where $k = c$, this expression reduces to $\mathbb{E}[X^c] = \lambda^c$ because $\Gamma(2) = 1$. This result will be usefull to asses the performance of our estimator afterwards.

(3.b)

Using a nonparametric bootstrap method, the bias and the variance of our proposed estimator are :

$$\text{Bias}^*(T) \approx \frac{1}{B}\sum_{b=1}^{B} T(\mathcal{X}^{*(b)}) - T(\mathcal{X}) = -0.04$$

$$\text{Var}^*(T) \approx \frac{1}{B}\sum_{b=1}^{B} T(\mathcal{X}^{*(b)})^2 - \left(\frac{1}{B}\sum_{b=1}^{B} T(\mathcal{X}^{*(b)})\right)^2 = 24.3$$

We see that the Bias[4] is -0.04, indicating that our estimator slightly underestimates the true value of the parameter. The Variance is 24.3, reflecting the variability of the estimator across different bootstrap samples. To obtain those results, we first simulated an initial sample of size $n = 30$ from a Weibull distribution, with scale parameter $\lambda = 5$ and shape parameter $c = 2$. Then, we generated $B = 10,000$ bootstrap pseudo-samples[5] of size n, and computed the estimators $T(\mathcal{X}^{*(b)})$.

[4] We do not correct the Bias as $|\text{Bias}^*(T)|/\text{Std}^*(T) \leq 0.25$

[5] The choice of B as been done based on the value of the error bound. For $B = 10,000$, the error bound is equal to 0.096

Regarding the existence of an asymptotic CI, based on the Central Limit Theorem, we can assume that :

$$T(\mathcal{X}) - \mathbb{E}[X^c] \xrightarrow{\mathcal{D}} N(0, \sigma^2(F))$$

This asymptotic result relies on the assumption that there is a sufficiently large sample size ($n \geq 30$). Otherwise, the normal approximation might be poor.

The **asymptotic CI** for $\mathbb{E}[X^c]$ is given by :

$$\left[T(\mathcal{X}) - \hat{\sigma}(\mathcal{X})z_{1-\alpha/2}, T(\mathcal{X}) + \hat{\sigma}(\mathcal{X})z_{1-\alpha/2}\right] = [17.08; 36.73]$$

Where $\hat{\sigma}(\mathcal{X})$ is some estimator of $\sigma(F)$ and z a quantile of the standard Normal distribution.

Regarding now the **bootstrap CIs**, we computed three quantities :

(1) The bootstrap estimates $T(\mathcal{X}^{*(b)})$

(2) The pivotal quantity $T(\mathcal{X}^{*(b)}) - T(\mathcal{X})$ (as here $\theta(F_n) = T(\mathcal{X})$)

(3) The studentized quantity $n^{1/2} \cdot \frac{T(\mathcal{X}^*) - T(\mathcal{X})}{\hat{\sigma}(\mathcal{X}^*)}$

Based on the 2 first quantities, the **basic bootstrap CI** is given by :

$$\left[T(\mathcal{X}) - u^*_{1-\alpha/2}; T(\mathcal{X}) - u^*_{\alpha/2}\right] = [17.63; 36.43]$$

Where u^*_a is the a-quantile of quantity (2).

The **Percentile Bootstrap CI** on the other hand is given by :

$$\left[T(\mathcal{X}) + u^*_{\alpha/2}; T(\mathcal{X}) + u^*_{1-\alpha/2}\right] = [17.38; 36.18]$$

It relies on the assumption that there exists an (unknown) monotone transformation $U = h(T)$, of $T(\mathcal{X}) = \theta(F_n)$, which has a symmetric distribution around $h(\theta)$.

For the **t-Bootstrap CI**, we used the third quantity calculated and leveraged double bootstrap to estimate $\sigma^2(F)$, leading to the following CI :

$$\left[T(\mathcal{X}) - n^{-1/2}\hat{\sigma}(\mathcal{X})y^*_{1-\alpha/2}; T(\mathcal{X}) - n^{-1/2}\hat{\sigma}(\mathcal{X})y^*_{\alpha/2}\right] = [17.9; 41]$$

Where y^*_a is the a-quantile of quantity (3).

Method	Lower Bound	Upper Bound
Asymptotic CI	17.08325	36.72813
Bootstrap Basic CI	17.62776	36.42648
Bootstrap Percentile CI	17.38489	36.18361
Iterated t-Bootstrap CI	17.90901	41.03581

Table 1: 95 % Confidence intervals for the different methods seen in class.

All the CIs contain the true value of $\mathbb{E}[X^c]$ which, for our simulation, is equal to $\lambda^c = 25$. The different CI methods have almost the same results, with only slight differences. However, we observe that the t-bootstrap CI is wider than the others. This method uses a second bootstrap which adds an additional layer of uncertainty. That could explain why the CI is larger than the others.

(3.c)

We aim to test two hypotheses regarding the parameter of interest $\mathbb{E}[X^c]$. As $\mathbb{E}[X^c] = \lambda^c$, the first hypothesis we chose is $H_0^T : \mathbb{E}[X^c] = 5^2 = 25$, while the second hypothesis we chose is $H_0^F : \mathbb{E}[X^c] = 4^2 = 16$, and is not satisfied under the model.

These are the p-values we obtain for the hypotheses :

$$\text{P-value}_{H_0^T} = 0.48$$
$$\text{P-value}_{H_0^F} = 0.03$$

These p-values imply that we do not reject the true hypothesis H_0^T and that we do reject the false null hypothesis H_0^F.

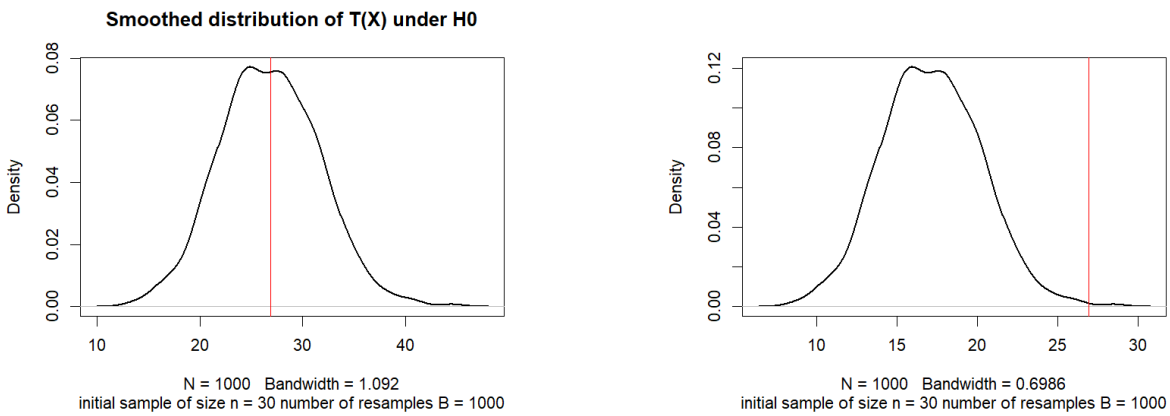

Figure 3: Estimated densities of the estimator under H_0 using bootstrap. On the right panel, the density corresponds to the one under H_0^T, while the one on the right panel corresponds to the density under H_0^F. The observed $T(\mathcal{X})$ is represented with a vertical red line.

To be more specific about the process, we first simulated an original sample $\mathcal{X} = X_1, \ldots, X_{30}$, where $X_i \sim \text{Weibull}(5, 2)$, and computed the observed statistic $T(\mathcal{X})$. To test whether we should reject H_0, we generate resamples $\mathcal{X}^{*(b)}$ from a sample $\mathcal{X}_{H_0}$ that satisfies the Weibull distribution assumption under the null. Then, we compute the corresponding statistics $T(\mathcal{X}^{*(b)})$. This allowed us to compare the observed statistic with the resampled distribution and to calculate the empirical p-value, which, in the end, allowed us to determine whether the observed value is likely under H_0.

(3.d)

To assess the performance of the methods seen in class, we now calculate the **coverage probability** (*i.e.* the proportion of intervals containing the true parameter $\mathbb{E}[X^c] = 25$) and the **average width** of the intervals. The results are then compared across different sample sizes to analyze the impact of n on the accuracy and efficiency of the methods :

Method	$n = 10$	$n = 50$	$n = 100$	$n = 500$
Asymptotic	28.38 / 0.847	13.65 / 0.940	9.76 / 0.941	4.39 / 0.959
Basic Boot.	26.21 / 0.809	13.35 / 0.929	9.62 / 0.937	4.34 / 0.949
Perc. Boot.	26.21 / 0.843	13.35 / 0.938	9.62 / 0.939	4.34 / 0.952
t-bootstrap	45.79 / 0.934	14.79 / 0.956	10.12 / 0.943	4.40 / 0.954

Table 2: Performance of confidence intervals for $\mathbb{E}[X^c] = 25$: each cell displays the average length of the intervals and coverage probability (Length / Coverage). Nominal level is 95% over $M = 1000$ Monte-Carlo experiments, and for the bootstrap, $B = 500$.

For smaller sample sizes ($n = 10$), the t-bootstrap stands out for its ability to maintain a coverage close to 95% while keeping the interval width large. As the sample size increases ($n = 50$ and $n = 100$), all method becomes more competitive, with narrower intervals and coverage approaching the nominal 95% level.